

CNN do dado ao deploy — Nível 4, Caminho A

Arthur Callegari(RM565476) - Enos Barros(RM561926) · Lucas Zappia (565076) - 2TSCPV-2
Prof. Alexandre Carvalho · 24/08/2026

1. Problema

Montei o pipeline completo de visão computacional no Fashion-MNIST (70.000 imagens de roupas, 28×28, cinza) e depois troquei pelo CIFAR-10 (60.000 imagens coloridas, 32×32, com animais e veículos). Os dois têm 10 classes.

A pergunta que quis responder não foi só quanto a acurácia cai, mas por que ela cai mesmo com a rede ficando maior.

2. Pipeline

O código é separado por etapa: `config.py`, `data.py`, `model.py`, `train.py`, `evaluate.py`, `export_model.py` e `deploy/api.py`. Treinei no Colab com GPU T4. O serviço de inferência roda separado, porque o `api.py` não importa nada de `src/` — ele só precisa de `modelo_scriptado.pt` e `classes.json`.

2.1 Por que convolução e não camada densa

A convolução usa conectividade local: cada filtro olha só uma janela 3×3 por vez. E faz compartilhamento de pesos, ou seja, o mesmo filtro passa por todas as posições. Isso dá invariância a translação — o objeto pode estar em qualquer canto que a rede reconhece igual.

Uma camada densa aprenderia um peso separado para cada posição. No exercício 1.2 isso deu 4.640 parâmetros na convolução contra quase 20 milhões na densa equivalente.

2.2 O laço de treino

São cinco passos: `forward`, `perda`, `zero_grad`, `backward` e `step`. O `zero_grad` é o que mais me chamou atenção, porque o PyTorch acumula gradiente por padrão — se esquecer dele, o gradiente de um lote vira a soma de todos os anteriores e o treino desanda sem dar erro. Na validação uso `modelo.eval()` e `torch.no_grad()`, sem `backward` nem `step`.

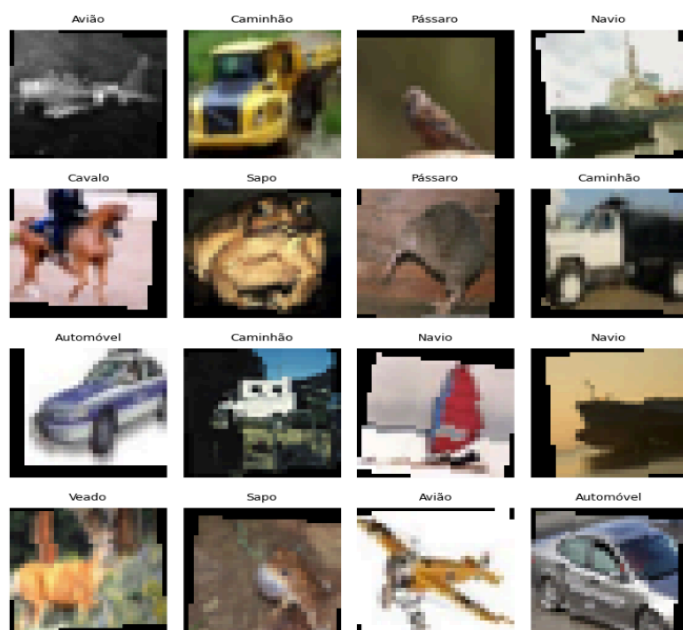
2.3 Dados e augmentation

Tirei a validação de dentro do treino (10%), ficando 45.000 / 5.000 / 10.000 no CIFAR-10. O teste foi usado uma única vez, no final, e nenhum hiperparâmetro foi escolhido olhando para ele.

Uma coisa que não percebi no começo: usar `random_split` direto faria treino e validação compartilharem a mesma transformação, e a validação receberia `augmentation`. O código resolve carregando o dataset duas vezes e aplicando os mesmos índices com `Subset`.

No treino uso espelhamento horizontal, recorte aleatório e rotação pequena; validação e teste só `ToTensor` e `Normalize`. O espelhamento horizontal vale no CIFAR-10 (carro espelhado ainda é carro), mas o vertical seria errado.

Amostras do treino do CIFAR-10 depois do `augmentation`:



3. O que mudou na arquitetura

Parâmetro	Fashion-MNIST	CIFAR-10
Canais de entrada	1 (cinza)	3 (RGB)
Resolução	28 × 28	32 × 32
Média	0,2860	(0,4914 · 0,4822 · 0,4465)
Desvio	0,3530	(0,2470 · 0,2435 · 0,2616)
Padding do RandomCrop	2	4
Flatten	576 (64×3×3)	1024 (64×4×4)
Parâmetros	98.554	156.186

As estatísticas de normalização do CIFAR-10 são três, uma por canal, calculadas sobre o treino. Reaproveitar as do Fashion-MNIST faria o modelo convergir pior sem dar erro nenhum.

Os formatos saem da fórmula $\lfloor (entrada + 2 \cdot padding - kernel) / stride \rfloor + 1$: $28 \rightarrow 14 \rightarrow 7 \rightarrow 3$ no Fashion-MNIST ($7 \div 2$ vira 3, não 3,5) e $32 \rightarrow 16 \rightarrow 8 \rightarrow 4$ no CIFAR-10.

O aumento de 98.554 para 156.186 parâmetros vem quase todo da primeira camada densa, que passou de 576×128 para 1024×128 — 57.344 pesos a mais. A parte convolucional só ganhou 288. Camada densa é cara, convolução é barata. E a ironia: o modelo ficou 58% maior e vai acertar bem menos.

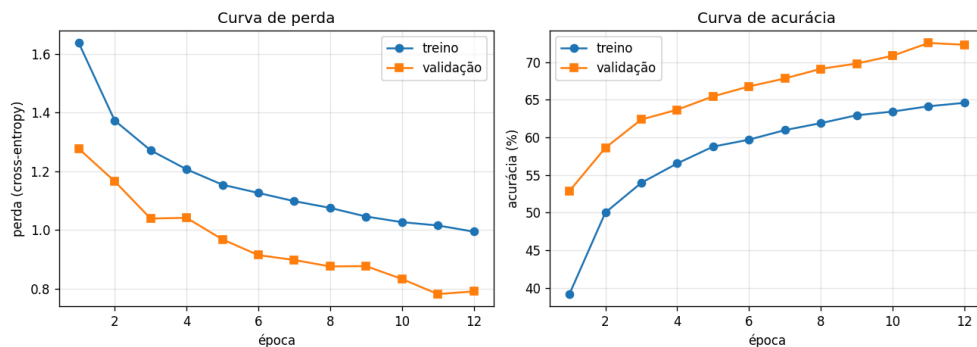
4. Resultados

Métrica	Fashion-MNIST	CIFAR-10	Diferença
Validação (melhor)	90,32%	72,56%	-17,76 pp
Teste	89,74%	71,67%	-18,07 pp
Perda no teste	0,2793	0,8057	—
Top-2	97,42%	87,53%	-9,89 pp
Parâmetros	98.554	156.186	+58%

A queda entre validação e teste foi pequena nos dois casos (0,58 e 0,89 ponto), o que indica que não houve vazamento nem ajuste em cima do teste. Sobre reprodutibilidade: o professor reportou 89,57% e eu cheguei em 89,74% com a mesma semente — a diferença vem de eu ter rodado em GPU e ele em CPU, porque o cuDNN não é totalmente determinístico.

4.1 Curvas

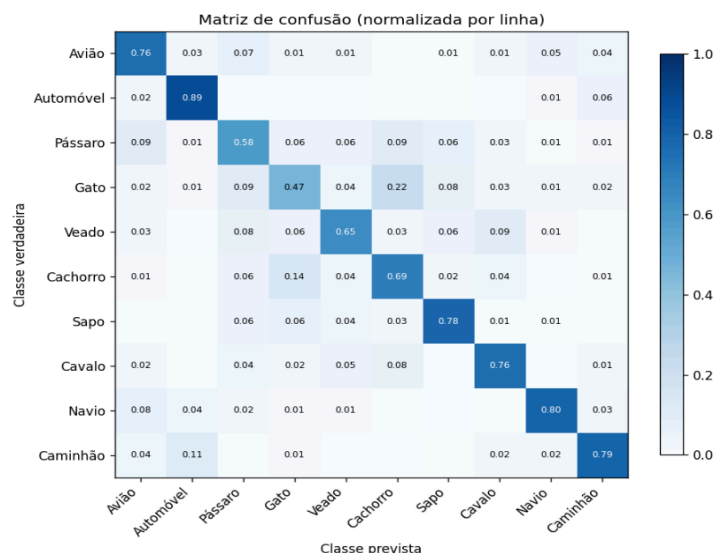
Curvas de perda e acurácia do CIFAR-10:



Eu esperava overfitting no CIFAR-10, mas não foi o que aconteceu: as duas curvas continuaram subindo até a época 12. Isso é underfitting — o treino parou por falta de tempo, não porque tinha aprendido tudo. Duas evidências: o early stopping nunca disparou e a taxa de aprendizado nunca foi reduzida (no Fashion-MNIST ela caiu na época 10 e a acurácia subiu logo depois, de 88,95% para 90,25%).

Nos dois datasets a validação ficou acima do treino. Isso não é bug: o augmentation só é aplicado no treino e o dropout desliga 30% dos neurônios nesse modo. A diferença foi de uns 2 pontos no Fashion-MNIST e uns 8 no CIFAR-10, ou seja, a regularização pesa mais no problema difícil.

4.2 Matriz de confusão

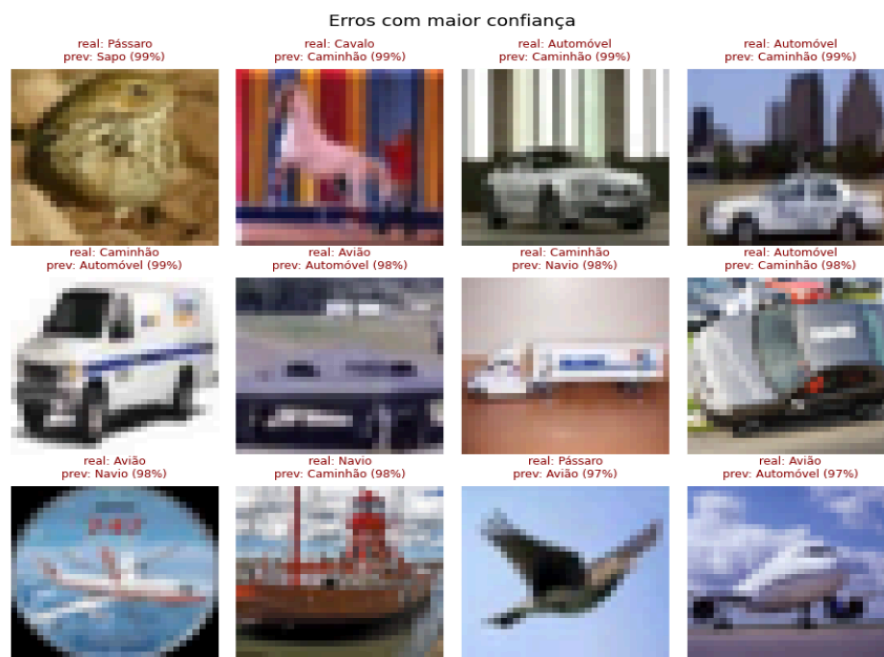


Verdadeiro	Previsto	Ocorrências	Observação
Gato	Cachorro	223	Maior confusão
Cachorro	Gato	138	O par inverso
Caminhão	Automóvel	114	Único par forte entre veículos
Pássaro	Avião	93	Os dois aparecem no céu

As três piores classes são animais (Gato F1 0,51, Pássaro 0,58, Cachorro 0,64) e as três melhores são veículos (Automóvel 0,84, Navio 0,83, Caminhão 0,80). Acho que é porque veículos têm forma rígida, enquanto um gato pode estar deitado, sentado ou de costas.

O Gato é o pior caso: revocação de 0,467, e aparece em quatro dos oito pares mais confundidos. No Fashion-MNIST o problema equivalente era a Camisa (0,632), mas lá 92% dos erros iam para só três vizinhas.

4.3 Análise dos erros: Os doze erros com maior confiança



Essa figura foi a que mais me chamou atenção. Um pássaro de asas abertas no céu virou Avião com 97%; um avião sobre superfície escura virou Navio com 98%; um cavalo em frente a um fundo colorido virou Caminhão com 99%.

Minha conclusão é que o modelo usa o fundo como atalho. Em 32×32 o fundo ocupa mais área que o objeto e correlaciona bem o suficiente durante o treino. Os filtros da primeira camada reforçam isso: no Fashion-MNIST parecem detectores de borda, e no CIFAR-10 são só manchas de cor.

5. Por que o CIFAR-10 é mais difícil

Juntando os resultados, acho que são quatro motivos. Primeiro, a variação dentro da mesma classe: uma calça é sempre vista de frente, um gato assume infinitas poses. Segundo, o fundo: no Fashion-MNIST ele é preto puro e não carrega informação, então a rede é obrigada a olhar a silhueta; no CIFAR-10 ele tem informação, mas engana.

Terceiro, a cor: um caminhão pode ser amarelo, branco ou vermelho e todos são Caminhão. Quarto, o que define a classe: no Fashion-MNIST é o formato, no CIFAR-10 é o significado — gato e cachorro têm formato parecidíssimo em 32×32 .

Além disso, quadruplicar os filtros rendeu 1,03 ponto no Fashion-MNIST e 5,18 no CIFAR-10. No primeiro a rede já tinha tirado quase tudo do dado; no segundo ainda tinha espaço.

6. Tabela de experimentos

Todos com 5 épocas, mudando uma variável por vez. Comparação por validação, porque o teste é usado uma vez só.

Experimento	Fashion val	CIFAR val	Parâmetros (CIFAR)	Tempo/ép.
baseline (5 ép.)	88,42%	65,66%	156.186	25 s
sem normalização	88,18%	61,58%	156.186	—
sem augmentation	90,95%	70,94%	156.186	14 s
canais (8,16,32)	85,87%	59,24%	73.042	24 s
canais (32,64,128)	89,45%	70,84%	357.034	25 s
lr = 1e-1	10,78%	19,14%	156.186	24 s
lr = 1e-5	73,63%	34,94%	156.186	24 s
dropout 0,0	88,72%	68,66%	156.186	24 s
dropout 0,7	82,97%	52,52%	156.186	25 s
sem BatchNorm	86,57%	60,24%	155.962	25 s
sem BN, lr 1e-2	82,58%	35,16%	155.962	25 s

Duas coisas não saíram como eu esperava. Tirar o augmentation melhorou o resultado nos dois datasets — em 5 épocas ele só atrapalha o começo, e o benefício apareceria com mais épocas. E tirar a normalização quase não mudou nada no Fashion-MNIST, embora tenha custado 4 pontos no CIFAR-10.

7. Deploy

O modelo é exportado em TorchScript, que salva arquitetura e pesos juntos. A verificação de saída idêntica passou nos dois casos: 3,34e-06 no Fashion-MNIST (390 KB) e 9,54e-07 no CIFAR-10 (615 KB). O modelo é carregado uma vez só, no lifespan do servidor.

O pré-processamento é reescrito na API sem torchvision, então precisa dar o mesmo resultado do treino.

Verificação	Fashion-MNIST	CIFAR-10	Referência
Health check /saude	ok	ok	—
testar_api.py (20 imagens)	18/20	17/20	89,74% / 71,67%
Latência média	3,4 ms	4,2 ms	—

A acurácia da API bateu com a do evaluate.py nos dois casos. Com 20 imagens a margem é grande, mas se os canais estivessem trocados o resultado seria uns 20 ou 30%, não 85%.

Os três erros do CIFAR-10 na API foram Gato→Cachorro, Sapo→Veado e Avião→Veado, com confiança abaixo de 62%, enquanto os acertos ficaram acima de 95%. Um limiar 0,65 separaria os três para revisão manual.

8. Conclusão

A acurácia caiu de 89,74% para 71,67% mesmo com o modelo 58% maior, o que mostra que o gargalo não era tamanho de rede.

O que mais me surpreendeu foi descobrir que o modelo do CIFAR-10 aprendeu a usar o fundo como pista. Pássaro no céu virando avião não é erro de quem confunde formas, é erro de quem olhou para o lugar errado.

Também achei interessante que o modelo não está tão perdido quanto o número sugere: no top-2 ele chega a 87,53%, ou seja, na maioria das vezes a resposta certa está em segundo lugar.

Se fosse continuar, tentaria nesta ordem: mais épocas (o modelo ainda melhorava quando parei), mais capacidade (rendeu 5 pontos em 5 épocas) e só depois mexer no augmentation.